Estimativa de π por Monte Carlo em MPI mestre/escravo: escalabilidade forte, fraca e granularidade*

Bernardo Fioreze
bernardofioreze@gmail.com

Repositório: https://github.com/BernardoFioreze/monte-carlo-simulation-pi

1 Problema e modelo

Estimamos π pela razão de pontos sorteados dentro do quarto de círculo unitário ($\pi \approx 4\,m/N$, com m pontos dentro de N amostras). A versão paralela segue o modelo **mestre/escravo** (saco de tarefas) em C+MPI: o saco tem $\lceil N/G \rceil$ tarefas de G amostras cada (granularidade G lida da linha de comando). A *iniciativa é do escravo* — cada resultado devolvido já conta como pedido da próxima tarefa; o mestre (rank 0) recebe por ordem de chegada com `MPI_ANY_SOURCE`, grava cada resultado na sua posição de origem (mantendo a ordem) e encerra cada escravo com uma *tag* distinta quando o saco esvazia. Funciona para qualquer $P \geq 2$ e qualquer número de tarefas.

2 Metodologia de medição

Ambiente. Nós `atlantica` com Intel Xeon E5520 @ 2,27 GHz (2 sockets × 4 núcleos × 2 *threads* = 8 núcleos físicos / 16 *threads* lógicas por nó). Alocamos **4 nós exclusivos** (= 32 núcleos físicos, 64 *threads* lógicas): assim $P \leq 32$ usa 1 processo por núcleo físico e $P = 64$ usa *Hyper-Threading* (`mpirun --use-hwthread-cpus --map-by core`).

Protocolo. 10 repetições por configuração; reportamos *mediana, mínimo e máximo*. A região medida usa `MPI_Wtime()` no mestre, do início da distribuição ao último resultado (sem geração/leitura da entrada). A referência sequencial (T1) foi *saneada* antes de virar *baseline*: removemos o I/O por iteração da versão original (que a deixava ∼200× mais lenta) e trocamos `rand()` pela mesma RNG *xorshift64* dos escravos (∼3 ns/amostra); ela mede só o laço de amostragem, com as mesmas flags (`-O2`) do MPI — sem isso o *speedup* seria medido contra uma baseline artificialmente lenta. Entrada calibrada em $N = 5{,}5 \times 10^9$ para o sequencial cair em 30–60 s (mediana 54,42 s; mín/máx 53,26/54,47 s).

Equivalência. O `CHECK` (total de pontos dentro do círculo) é uma soma de inteiros: para (N, G) fixos ele é **idêntico para qualquer** P (4 319 715 214 em toda a escala forte), provando a corretude do paralelismo. Como cada tarefa usa semente determinística própria, o fluxo difere do sequencial; a equivalência com este é por tolerância em π: $|\pi_{\text{MPI}} - \pi_{\text{seq}}| \approx 10^{-6}$, abaixo do erro estatístico $\sim 1/\sqrt{N} \approx 1{,}3 \times 10^{-5}$.

*Processamento Paralelo e Distribuído. Cluster Atlântica (LAD).

Tabela 1: Escala forte. Mediana de 10 reps; *speedup* e eficiência sobre o sequencial (54,42 s), por escravo ($P-1$).

P	mediana (s)	escravos	speedup	efic.
2	54,75	1	0,99	0,99
4	19,20	3	2,83	0,94
8	8,21	7	6,63	0,95
16	5,52	15	9,85	0,66
32	2,84	31	19,2	0,62
64	1,75	63	31,1	0,49

Tabela 2: Escala fraca. $N_P = 5{,}5 \times 10^9(P-1)$, $G = 10^6$. Eficiência *por escravo*: vazão por escravo $N_P/[(P-1)T]$ normalizada pela de P=2, que equivale a $T(2)/T(P)$ (trabalho por escravo constante ⇒ ideal 1).

P	mediana (s)	tarefas	efic.
2	54,52	5 500	1,00
4	57,58	16 500	0,95
8	57,52	38 500	0,95
16	82,76	82 500	0,66
32	83,26	170 500	0,65
64	84,52	346 500	0,65

3 Resultados

3.1 Escalabilidade forte (N fixo, $G = 10^6$)

Como o mestre não calcula, a aceleração é referida aos $P-1$ escravos (Tab. 1, Fig. 1a). A eficiência é quase ideal *dentro de um nó* (∼ 0,95 até $P = 8$, pois 8 processos cabem nos 8 núcleos de um nó) e **cai em** $P = 16$ (0,66), quando o cálculo cruza para um segundo nó e cada mensagem mestre↔escravo passa a pagar latência de rede.

3.2 Escalabilidade fraca (trabalho por escravo constante)

No mestre/escravo só $P-1$ processos calculam, então crescemos $N \propto (P-1)$ (e não $\propto P$) para manter o trabalho por escravo fixo em $\approx 5{,}5 \times 10^9$ amostras. A *medida de trabalho* é determinística: o nº de tarefas é exatamente $5500\,(P-1)$; o `CHECK` acompanha $\propto (P-1)$ dentro do ruído estatístico. O tempo ideal é constante. A curva (Tab. 2, Fig. 1b) fica **plana em ∼57 s até** $P = 8$ (um nó), dá um **degrau para ∼83 s em** $P = 16$ (cruza nós) e estabiliza — o *mesmo* efeito de fronteira de nó da escala forte. Como a eficiência é medida *por escravo* (vazão por escravo, ideal 1), nenhum valor excede 1.

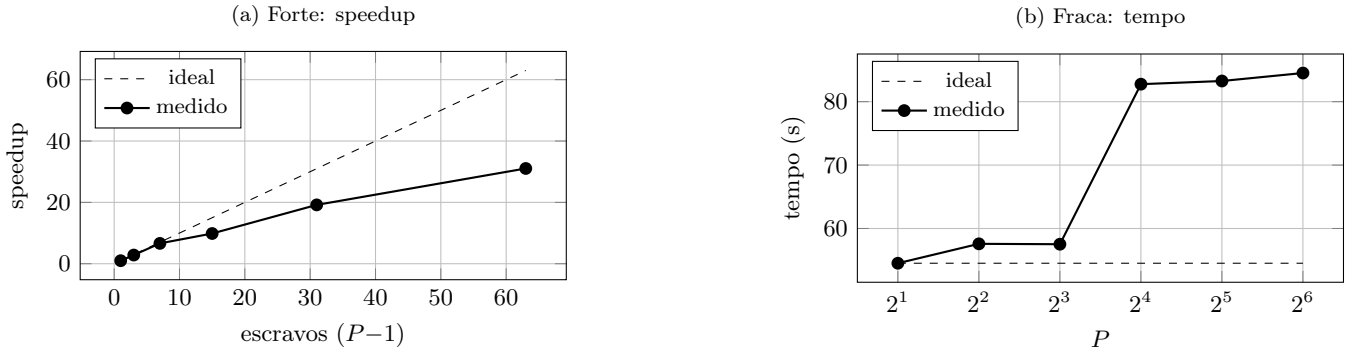


Figura 1: Escalabilidade. (a) o *speedup* descola do ideal a partir de P=16; (b) o tempo da escala fraca é plano dentro de um nó e sobe um degrau ao cruzar nós.

Tabela 3: Granularidade, $P = 32$, $N = 5{,}5 \times 10^9$.

G	mediana (s)	tarefas	regime
10^4	5,23	550 000	comunicação
10^6	2,85	5 500	ótimo
10^8	3,90	55	desbalanceamento

3.3 Granularidade e balanceamento ($P = 32$)

Variando G com $P = 32$ (Tab. 3) surge o U clássico: G pequeno (550 000 tarefas) satura o mestre de mensagens minúsculas; G grande (55 tarefas para 31 escravos) causa desbalanceamento (escravos ociosos no fim). O ótimo é $G = 10^6$. **Consistência entre tabelas**: esse ponto (2,85 s) coincide com a forte em $P = 32$ (2,84 s).

4 Discussão: previsão do gargalo

O custo de uma tarefa é $t_{\text{tar}} \approx G \cdot t_{\text{am}}$, com $t_{\text{am}} = 54{,}42\,\text{s}/5{,}5 \times 10^9 \approx 9{,}9\,\text{ns}$, logo $t_{\text{tar}}(G{=}10^6) \approx 9{,}9\,\text{ms}$. O mestre satura quando o nº de escravos excede $t_{\text{tar}}/t_{\text{msg}}$. Como o tempo da forte *ainda cai* de $P = 32$ para $P = 64$, o mestre **não saturou** até 63 escravos, o que impõe $t_{\text{msg}} \lesssim t_{\text{tar}}/63 \approx 160\,\mu\text{s}$ — coerente com latência de rede. A medição direta de t_{msg} (ping-pong) fecharia a previsão quantitativamente. Todos os três experimentos apontam a *mesma* causa: a comunicação mestre↔escravo ao cruzar a fronteira de nó, e não a computação, é o limite de escalabilidade.

5 Conclusão

O modelo mestre/escravo escala de forma quase ideal dentro de um nó e é limitado pela comunicação ao usar múltiplos nós. A escala fraca redefinida como $N \propto (P{-}1)$ elimina o artefato do processo coordenador e produz eficiência *por escravo* ≤ 1; a granularidade ótima ($G = 10^6$) equilibra comunicação e balanceamento. A corretude é comprovada pelo `CHECK` invariante em P.

6 Uso de inteligência artificial

Utilizou-se assistência de IA (Claude, Anthropic) na implementação do código MPI mestre/escravo, nos scripts de medição (`sbatch`) e na redação deste relatório. As decisões de modelagem, a execução das medições no cluster e a validação dos resultados foram conduzidas e conferidas pelos autores, que respondem por cada chamada MPI e por cada número apresentado.

Referências

[1] MPI Forum. *MPI: A Message-Passing Interface Standard*, v4.1, 2023.

[2] Repositório do trabalho (fontes, scripts `sbatch`, saídas brutas e este relatório): `https://github.com/BernardoFioreze/monte-carlo-simulation-pi`.